2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 01

Жизненный цикл программы и модели исполнения



Вспомнить всё

Общая схема трансляции:
Исходный код → препроцессор → компилятор → ассемблер → объектный файл → компоновщик → исполняемый файл → загрузчик → процесс в памяти.

Вспомнить всё

Общая схема трансляции:
Исходный код → препроцессор → компилятор → ассемблер → **объектный файл** → компоновщик → исполняемый файл → загрузчик → процесс в памяти.

Определение

Объектный файл (object file, объектный модуль) — это файл, содержащий машинный код и данные, сгенерированные компилятором или ассемблером из исходного кода, в форме, пригодной для последующей обработки компоновщиком и загрузчиком.

Ключевые свойства:

1. Неисполняемость (адреса отсчитываются от нуля).
2. Модульность (соответствует одной единице трансляции).
3. Самодокументируемость (содержит метаданные для компоновщика).
4. Разделение кода и данных (разные права доступа).

Но зачем?



kolorcuk • 6y ago

Давным-давно компьютеры были не такими крутыми, как сейчас. Скомпилировать все файлы проекта одним шагом было просто нереально - не хватало ни памяти, ни процессорной мощности, чтобы это обработать.

Чтобы сделать возможной компиляцию больших программ, был разработан промежуточный шаг в виде объектных файлов. Компиляторы сначала генерируют машинный код из одного маленького .c исходного файла за один шаг. Затем линковщик берет все объектные файлы и связывает их вместе. Это позволило писать компиляторы в 70-х годах и сделать возможным компилировать очень-очень большие программы, просто используя много .c исходных файлов.

Три формы объектных файлов

1. *Перемещаемый объектный файл* (relocatable object file). Содержит код и данные для объединения с другими файлами. Создаётся компилятором и ассемблером.
2. *Исполняемый объектный файл* (executable object file). Готов к загрузке в память и исполнению. Создаётся компоновщиком.
3. *Разделяемый объектный файл* (shared object file). Загружается в память и связывается динамически во время загрузки или выполнения.

Форматы объектных файлов

В зависимости от ОС:

- a.out — ранние версии Unix.
- PE (Portable Executable) — Windows (используется также в .NET).
- Mach-O — macOS, iOS.
- ELF (Executable and Linkable Format) — Linux, Unix, x86-64.

[!] Конкретный формат определяет детали кодирования, но базовые идеи (секции, таблица символов, записи перемещения) универсальны.

Структура ELF-файла

- Заголовок ELF (ELF header).
- Секции .text, .rodata, .data, .bss, .symtab, .rel.text, .rel.data, .debug, .line, .strtab.
- Таблица заголовков секций (Section header table).

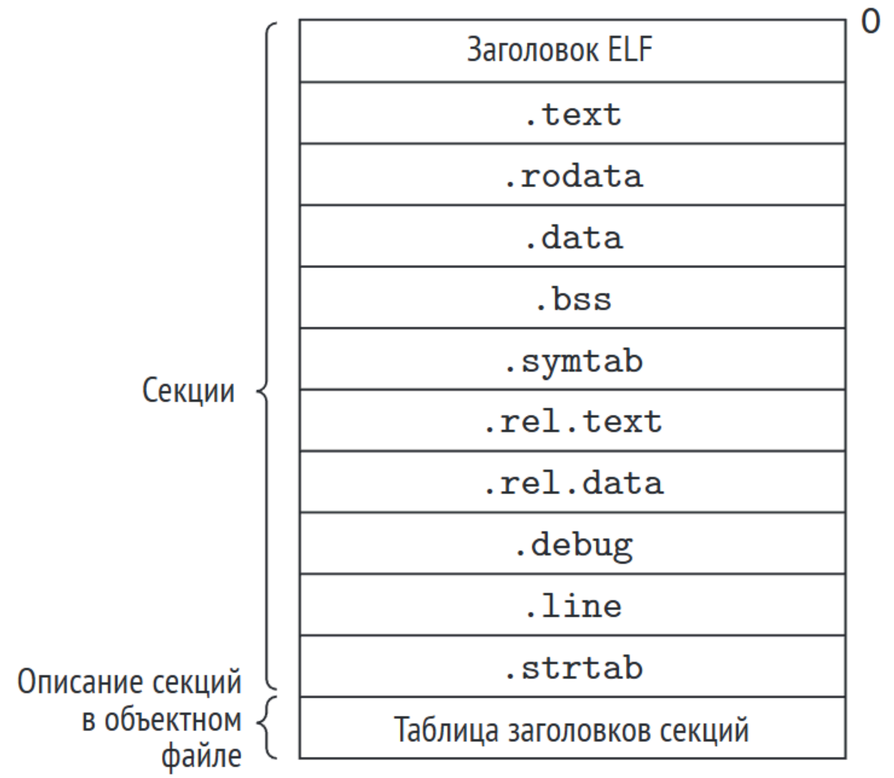


Рис. 7.2. Типичный перемещаемый объектный файл ELF

Заголовок ELF

Содержимое заголовка:

- Магическое число: 16-байтная последовательность, начинающаяся с 0x7f 'E' 'L' 'F'.
- Размер слова (32/64 бит) и порядок байтов (endianness).
- Тип объектного файла (перемещаемый, исполняемый, разделяемый).
- Архитектура целевого процессора (например, x86-64).
- Смещение, размер и количество записей таблицы заголовков секций.

```
1   readelf -h main.o
```

Секции кода и констант

- **.text**: Машинный код скомпилированной программы. Доступна только для чтения и исполнения (read + execute).
- **.rodata**: Данные только для чтения. Строки формата (printf), таблицы переходов (switch), константные массивы.

```
readelf -S main.o
```

Секции данных

- **.data**: Инициализированные глобальные переменные.
- **.bss**: Неинициализированные глобальные переменные.

- Фактически не занимает места в объектном файле на диске.
- Служит “заполнителем места”.
- При загрузке ОС выделяет память и заполняет её нулями.
- Мнемоника: bss — Better Save Space.
- Локальные переменные функций размещаются в стеке и не представлены в .data или .bss.

Служебные секции

- **.symtab**: Таблица символов (функции, глобальные переменные). Присутствует всегда (если не удалена через strip). Не содержит локальных переменных функций.
- **.rel.text**: Записи перемещения для кода (вызовы внешних функций, ссылки на глобальные переменные).
- **.rel.data**: Записи перемещения для инициализированных данных.
- **.debug** и **.line**: Отладочная информация и соответствие строк кода инструкциям. Добавляются только при компиляции с флагом -g.
- **.strtab**: Таблица строк для .symtab и .debug.

Безопасность и права секций

- Разделение секций: .text (read + execute), .data (read + write).
- Политика W^X (Write XOR Execute): область памяти не может быть одновременно доступна для записи и для исполнения.
- Механизмы DEP/NX (Data Execution Prevention / No-Execute).
- Предотвращение атак типа buffer overflow (внедрение кода в стек или кучу).

Три вида символов

С точки зрения компоновщика:

a. Глобальные символы, определённые в данном модуле. (Нестатические функции и глобальные переменные).
b. Внешние символы. (Глобальные имена, на которые ссылается модуль, но определённые в других модулях).
c. Локальные символы. (Определены и используются исключительно внутри модуля. Статические функции и переменные с атрибутом static).

Важно: Локальные символы для компоновщика — это не локальные переменные функций (которые размещаются в стеке).

Статические локальные переменные

```
1  int f() { static int x = 0; return x; }
2  int g() { static int x = 1; return x; }
```

- Размещаются не в стеке, а в .data или .bss.
- Компилятор создаёт уникальные локальные имена для компоновщика (например, x.1 и x.2).
- Каждая функция работает со своей переменной, несмотря на одинаковое имя в исходном коде.

Структура записи таблицы символов

Структура Elf64_Symbol:

- name — смещение строки имени в .strtab.
- type — тип объекта: функция (FUNC) или данные (OBJECT).
- binding — привязка: локальное (LOCAL) или глобальное (GLOBAL).
- section — индекс секции. Специальные значения: ABS (неперемещаемые), UND (неопределённые/внешние), COMMON.
- value — смещение от начала секции (для перемещаемых) или абсолютный адрес (для исполняемых).
- size — размер объекта в байтах.

Пример: Конвейер компиляции в Go

Схема:
Исходный код (пакет) -> go tool compile -> Объектный файл .o (собственный формат Go) -> go tool link -> Исполняемый бинарник.

Компилятор Go использует собственный формат для промежуточных объектных файлов, оптимизированный под нужды компоновщика Go. ELF порождается только на финальном этапе. Концепция объектного файла первична, формат вторичен.

Пример: Релокация в Go

```
1   func main() { fmt.Println("Hello") }
```

- В ассемблерном листинге вызов fmt.Println имеет смещение.
- В выводе go tool objdump присутствует идентификатор R_CALL.
- В выводе go tool nm символ fmt.Println помечен флагом U (undefined).

Вычисление смещения компоновщиком:
Адрес(fmt.Println) - Адрес(main) - Смещение_внутри_main - Размер_инструкции = Относительное_смещение

Инструменты анализа объектных файлов

Пакет GNU binutils:

- ar — управление статическими библиотеками.
- strings — вывод печатаемых строк (преимущественно из .rodata).
- strip — удаление таблицы символов.
- nm — перечисление символов из .symtab.
- size — размеры секций.
- readelf — полная структура ELF-файла.
- objdump — дизассемблирование и полная информация.

Итоги

- Объектный файл — структурированный контейнер, содержащий код, данные, метаданные и служебную информацию.
- Формат ELF является стандартом для x86-64; базовые принципы универсальны для всех форматов, включая собственные форматы современных языков (Go).
- Секции .text, .rodata, .data, .bss разделяют код и данные по назначению и правам доступа.
- Секция .bss не занимает места в файле, экономя дисковое пространство.
- Таблица символов .symtab содержит информацию о глобальных именах: функциях, переменных, внешних символах.
- Инструменты readelf, nm, objdump позволяют анализировать объектные файлы без исходного кода.